Cookie Puzzle

Aunt Gertrude knows that one of Mrs. Brown, Cecilia, Danielle or Eduardo ate all the cookies. She also knows that the guilty person always lies and the innocent always tells the truth.

- Brown says: "The culprit is either Eduardo or Danielle."
- Eduardo says: "The guilty person is a girl."
- Finally, Danielle says: "If Brown is guilty, then Cecilia is innocent."

Who ate the cookies? Who is guilty and who is innocent?

Predicate Logic Solution

To determine who ate the cookie, we solve the problem using **predicate logic**. The solution follows all necessary steps, including defining the domain, translating statements into logical formulas, proving consistency, and concluding with the transformation of the knowledge base and the application of the **resolution method** to derive the final conclusion.

Contents

1. **Vocabulary**
   - Constants
   - Domains
   - Variables
   - Predicates
2. **Knowledge Base Formalization**
3. **CNF Transformation**
4. **Knowledge Base Clauses**
5. **First-Order Resolution with Set-of-Support Strategy**
   - 5.1 Danielle
   - 5.2 Brown
   - 5.3 Eduardo
   - 5.4 Cecilia
   - 5.5 Brown
6. **Automated Solver Verification**
   - 6.1 CPLJ All Solver
   - 6.2 CP4J Program
7. **Conclusion**

1. Vocabulary

Based on the problem statement, the main elements of the formal language can be identified: constants, variables, and predicates.

Constants

Constants represent the specific individuals mentioned in the problem.

Object	Constant	Information
Ann	a	female
Bruno	b	male
Cecilia	c	female
Danielle	d	female
Eduardo	e	male

Domains

To ensure the rules address only the relevant individuals, we define the set of all suspects:

$$D = \{a, b, c, d, e\}$$

Variables

x, y, z : Elements of the domain D (e.g., suspects).

Predicates

- $\text{guilty}(x)$: x participates in a lie if and only if the individual x is guilty.
- $\text{girl}(x)$: x is an individual x is female.

2. Knowledge Base Formalization

- **At least one of Mrs. Brown, Cecilia, Danielle, or Eduardo is guilty:**

$$\text{guilty}(a) \vee \text{guilty}(b) \vee \text{guilty}(c) \vee \text{guilty}(d) \vee \text{guilty}(e)$$

- **Ann is a female:**

$$\text{girl}(a)$$

- **Bruno is a male:**

$$\neg\text{girl}(b)$$

- **Cecilia is a female:**

$$\text{girl}(c)$$

- **Danielle is a female:**

$$\text{girl}(d)$$

- **Eduardo is a male:**

$$\neg\text{girl}(e)$$

- **Bruno says: "The culprit is either Eduardo or Danielle":**

$$\text{guilty}(b) \leftrightarrow (\text{guilty}(e) \vee \text{guilty}(d))$$

- **Eduardo says: "The guilty person is a girl":**

$$\text{guilty}(e) \leftrightarrow (\exists\, \text{Z}\, (\text{girl}(\text{Z}) \wedge \text{girl}(\text{Z}))$$

- **Danielle says: "If Brown is guilty, then Cecilia is innocent":**

$$\text{guilty}(d) \leftrightarrow (\text{guilty}(b) \rightarrow \neg\text{guilty}(c))$$

3. CNF Transformation

- **At least one of Mrs. Brown, Cecilia, Danielle, or Eduardo is guilty:**

  Formula:

$$\text{guilty}(a) \vee \text{guilty}(b) \vee \text{guilty}(c) \vee \text{guilty}(d) \vee \text{guilty}(e)$$

  Resulting clauses:

$$\text{guilty}(a) \vee \text{guilty}(b) \vee \text{guilty}(c) \vee \text{guilty}(d) \vee \text{guilty}(e)$$

- **Ann is a female:**

  Formula:

$$\text{girl}(a)$$

  Resulting clauses:

$$\text{girl}(a)$$

- **Bruno is a male:**

  Formula:

$$\neg\text{girl}(b)$$

  Resulting clauses:

$$\neg\text{girl}(b)$$

- **Cecilia is a female:**

  Formula:

$$\text{girl}(c)$$

  Resulting clauses:

$$\text{girl}(c)$$

- **Danielle is a female:**

  Formula:

$$\text{girl}(d)$$

  Resulting clauses:

$$\text{girl}(d)$$

- **Eduardo is a male:**

  Formula:

$$\neg\text{girl}(e)$$

  Resulting clauses:

$$\neg\text{girl}(e)$$

- **Bruno says: "The culprit is either Eduardo or Danielle":**

  Formula:

$$\text{guilty}(b) \leftrightarrow (\text{guilty}(e) \vee \text{guilty}(d))$$

  Conversion to CNF:

$$\text{guilty}(b) \leftrightarrow (\text{guilty}(e) \vee \text{guilty}(d))$$
$$\neg\text{guilty}(b) \vee (\text{guilty}(e) \vee \text{guilty}(d)) \wedge \neg(\text{guilty}(e) \vee \text{guilty}(d)) \vee \text{guilty}(b)$$
$$(\neg\text{guilty}(b) \vee \text{guilty}(e) \vee \text{guilty}(d)) \wedge (\neg\text{guilty}(e) \wedge \neg\text{guilty}(d)) \vee \text{guilty}(b)$$
$$(\neg\text{guilty}(b) \vee \text{guilty}(e) \vee \text{guilty}(d)) \wedge (\neg\text{guilty}(e) \vee \text{guilty}(b)) \wedge (\neg\text{guilty}(d) \vee \text{guilty}(b))$$
$$(\neg\text{guilty}(b) \vee \text{guilty}(e) \vee \text{guilty}(d)) \wedge (\neg\text{guilty}(e) \vee \text{guilty}(b)) \wedge (\neg\text{guilty}(d) \vee \text{guilty}(b))$$

  Resulting clauses:

$$\neg\text{guilty}(b) \vee \text{guilty}(e) \vee \text{guilty}(d)$$
$$\neg\text{guilty}(e) \vee \text{guilty}(b)$$
$$\neg\text{guilty}(d) \vee \text{guilty}(b)$$

- **Eduardo says: "The guilty person is a girl":**

  Formula:

$$\text{guilty}(e) \leftrightarrow (\exists\, \text{Z}\, (\text{girl}(\text{Z}) \wedge \text{girl}(\text{Z}))$$

  Conversion to CNF:

$$\text{guilty}(e) \leftrightarrow (\exists\, \text{Z}\, (\text{girl}(\text{Z}) \wedge \text{girl}(\text{Z}))$$
$$\neg\text{guilty}(e) \vee (\exists\, \text{Z}\, (\text{girl}(\text{Z}) \wedge \text{girl}(\text{Z})) \wedge \neg(\exists\, \text{Z}\, (\text{girl}(\text{Z}) \wedge \text{girl}(\text{Z})) \vee \text{guilty}(e)$$
$$\neg\text{guilty}(e) \vee (\exists\, \text{Z}\, (\text{girl}(\text{Z}) \wedge \text{girl}(\text{Z})) \wedge (\forall\, \text{Z}\, \neg(\text{girl}(\text{Z}) \wedge \text{girl}(\text{Z})) \vee \text{guilty}(e)$$
$$\neg\text{guilty}(e) \vee (\exists\, \text{Z}\, (\text{girl}(\text{Z}) \wedge \text{girl}(\text{Z})) \wedge (\forall\, \text{Z}\, \neg\text{girl}(\text{Z}) \vee \neg\text{girl}(\text{Z})) \vee \text{guilty}(e)$$
$$\neg\text{guilty}(e) \vee (\text{girl}(\text{Z}) \wedge \text{girl}(\text{Z})) \wedge (\forall\, \text{Z}\, \neg\text{girl}(\text{Z}) \vee \neg\text{girl}(\text{Z})) \vee \text{guilty}(e)$$
$$\neg\text{guilty}(e) \vee \text{girl}(\text{Z}) \wedge \text{girl}(\text{Z}) \wedge \neg\text{girl}(\text{Z}) \vee \neg\text{girl}(\text{Z}) \vee \text{guilty}(e)$$
$$\neg\text{guilty}(e) \vee \text{girl}(\text{Z}) \wedge \text{girl}(\text{Z}) \wedge \neg\text{girl}(\text{Z}) \vee \neg\text{girl}(\text{Z}) \vee \text{guilty}(e)$$

  Resulting clauses:

$$\neg\text{guilty}(e) \vee \text{girl}(\text{Z})$$
$$\neg\text{guilty}(e) \vee \text{girl}(\text{Z})$$
$$\neg\text{guilty}(e) \vee \neg\text{girl}(\text{Z}) \vee \neg\text{girl}(\text{Z}) \vee \text{guilty}(e)$$

- **Danielle says: "If Brown is guilty, then Cecilia is innocent":**

  Formula:

$$\text{guilty}(d) \leftrightarrow (\text{guilty}(b) \rightarrow \neg\text{guilty}(c))$$

  Conversion to CNF:

$$\text{guilty}(d) \leftrightarrow (\text{guilty}(b) \rightarrow \neg\text{guilty}(c))$$

- Expand equivalences:

$$\text{guilty}(d) \leftrightarrow (\neg\text{guilty}(b) \vee \neg\text{guilty}(c)) \wedge \neg\text{guilty}(c)$$
$$\neg\text{guilty}(d) \vee (\neg\text{guilty}(b) \vee \neg\text{guilty}(c)) \wedge \neg(\neg\text{guilty}(b) \vee \neg\text{guilty}(c)) \vee \text{guilty}(d)$$
$$\neg\text{guilty}(d) \vee (\neg\text{guilty}(b) \vee \neg\text{guilty}(c)) \wedge (\neg\text{guilty}(b) \wedge \neg\text{guilty}(c)) \vee \text{guilty}(d)$$
$$\neg\text{guilty}(d) \vee (\neg\text{guilty}(b) \vee \neg\text{guilty}(c)) \wedge (\neg\text{guilty}(b) \vee \neg\text{guilty}(c)) \wedge (\neg\text{guilty}(c) \vee \neg\text{guilty}(c)) \vee \text{guilty}(d)$$
$$\neg\text{guilty}(d) \vee (\neg\text{guilty}(b) \vee \neg\text{guilty}(c)) \wedge (\neg\text{guilty}(b) \vee \neg\text{guilty}(c)) \wedge (\neg\text{guilty}(c) \vee \neg\text{guilty}(c)) \vee \text{guilty}(d)$$

  Resulting clauses:

$$\neg\text{guilty}(d) \vee \neg\text{guilty}(b) \vee \neg\text{guilty}(c)$$
$$\neg\text{guilty}(d) \vee \neg\text{guilty}(b) \vee \neg\text{guilty}(c)$$
$$\neg\text{guilty}(d) \vee \neg\text{guilty}(c) \vee \neg\text{guilty}(c)$$

4. Knowledge Base Clauses

After converting the formal representation of the knowledge base into CNF, we removed all duplicates and observed the final mathematical set of clauses.

No	Knowledge Base Clause
1	$\text{guilty}(a) \vee \text{guilty}(b) \vee \text{guilty}(c) \vee \text{guilty}(d) \vee \text{guilty}(e)$
2	$\text{girl}(a)$
3	$\neg\text{girl}(b)$
4	$\text{girl}(c)$
5	$\text{girl}(d)$
6	$\neg\text{girl}(e)$
7	$\neg\text{guilty}(b) \vee \text{guilty}(e) \vee \text{guilty}(d)$
8	$\neg\text{guilty}(e) \vee \text{girl}(\text{Z})$
9	$\neg\text{guilty}(e) \vee \text{girl}(\text{Z})$
10	$\neg\text{guilty}(e) \vee \neg\text{girl}(\text{Z}) \vee \neg\text{girl}(\text{Z}) \vee \text{guilty}(e)$
11	$\neg\text{guilty}(d) \vee \neg\text{guilty}(b) \vee \neg\text{guilty}(c)$
12	$\neg\text{guilty}(d) \vee \neg\text{guilty}(b) \vee \neg\text{guilty}(c)$
13	$\neg\text{guilty}(d) \vee \neg\text{guilty}(c) \vee \neg\text{guilty}(c)$

5. First-Order Resolution with Set-of-Support Strategy

The **first-order resolution method** is a powerful logical inference technique for proving the **first-order logic**, which allows reasoning about all elements in a domain. It operates on clauses in conjunctive normal form (CNF) and is enhanced by applying the **resolution rule** together with **unification** to eliminate variables and match complementary literals.

The **set-of-support resolution method** is a refinement of this approach, where resolution is applied only to clauses that involve a specific subset of the knowledge base, called the set of support. This strategy ensures that the reasoning process remains focused on the relevant hypotheses and avoids unnecessary derivations, and helps lead to contradiction or the desired conclusion more efficiently.

Core ideas:

- All clauses are divided into two sets:
  - **Knowledge Base (KB)** – trusted facts and rules.
  - **Set of Support (SoS)** – clauses related to the negation of the goal.

Resolution rule:

- Resolution is applied **only if at least one parent clause belongs to the SoS**.
- Unification is used to match variables across clauses in first-order logic.

Why it works:

- Reduces the search space.
- Prevents unnecessary resolutions.
- Guides reasoning toward a contradiction or proof.

Goal:

- Derive the **empty clause** $\square$, which confirms the conclusion.

Results:

- Enables consistent and efficient reasoning in **first-order logic**.

5.1 Danielle

First, we ask the hypothesis that **Danielle is innocent**: $\neg\text{guilty}(d)$. To do this, we apply the resolution method in order to confirm or refute the assumption.

No	Derived Clause	Resolution Step / Description
15	$\neg\text{guilty}(d)$	Negated conclusion
16	$\neg\text{guilty}(b) \vee \neg\text{guilty}(c) \vee \neg\text{guilty}(c)$	Resolution of Clauses 15 and 11
17	$\neg\text{guilty}(b) \vee \neg\text{guilty}(c)$	Tautology elimination of clause 16
18	$\neg\text{guilty}(c)$	Resolution of Clauses 17 and 17

As a result of the resolution process, the **empty clause** is derived, which indicates a logical contradiction when the negation of the hypothesis is assumed. Therefore, the statement "$\neg\text{guilty}(d)$ is true, and we can safely conclude that **Danielle is innocent**.

5.2 Brown

Knowing that **Danielle is innocent**, we can assume that her statement is true: **Brown is guilty**. Then **Cecilia is innocent**. Based on this, we add the hypothesis that Brown is innocent: $\neg\text{guilty}(b)$.

No	Derived Clause	Resolution Step / Description
19	$\neg\text{guilty}(b)$	Negated conclusion
20	$\neg\text{guilty}(b) \vee \neg\text{guilty}(c) \vee \neg\text{guilty}(c)$	Resolution of clauses 19 and 11
21	$\neg\text{guilty}(c)$	Tautology elimination of clause 20
22	$\square$	Resolution of Clauses 21 and 21

As a result of the resolution process, the **empty clause** is derived, which indicates a logical contradiction when the negation of the hypothesis is assumed. Therefore, the statement "$\neg\text{guilty}(b)$ is true, and we can safely conclude that **Brown is innocent**.

5.3 Eduardo

We already have proofs that **Danielle** and **Brown** are innocent. Brown claims that the guilty person is either **Eduardo** or **Danielle**. However, since we already know that **Danielle is innocent**, we can rule out the hypothesis that **Eduardo is guilty**: $\neg\text{guilty}(e)$.

No	Derived Clause	Resolution Step / Description
23	$\neg\text{guilty}(e)$	Negated conclusion
24	$\neg\text{guilty}(e) \vee \text{girl}(\text{Z})$	Resolution of clauses 23 and 8
25	$\neg\text{guilty}(e) \vee \neg\text{girl}(\text{Z}) \vee \neg\text{girl}(\text{Z}) \vee \text{guilty}(e)$	Unification: substitute Z/d in clause 12
26	$\neg\text{guilty}(e) \vee \neg\text{girl}(\text{Z})$	Resolution of clauses 25 and 24
27	$\neg\text{girl}(\text{Z})$	Resolution of clauses 26 and 23
28	$\square$	Resolution of Clauses 27 and 25

The resolution process starting from the negated hypothesis $\neg\text{guilty}(e)$ resulted in the derivation of an **empty clause**, which proves that **Eduardo is innocent**. Moreover, this result shows that the statement claiming that at least one of the girls is guilty is false.

5.4 Cecilia

We know that **Eduardo is guilty**, we have recorded the hypothesis as answer **number 18**. Now, in order to verify that **Cecilia is innocent**, we add the hypothesis: $\neg\text{girl}(c)$.

No	Derived Clause	Resolution Step / Description
29	$\neg\text{girl}(c)$	Negated conclusion
30	$\neg\text{guilty}(d) \vee \neg\text{guilty}(b) \vee \neg\text{guilty}(c)$	Resolution of Clauses 29 and 11
31	$\neg\text{guilty}(d) \vee \neg\text{guilty}(b) \vee \neg\text{guilty}(c)$	Tautology elimination of clause 30
32	$\square$	Resolution of Clauses 18 and 31

As a result of the resolution with the negation of the hypothesis that **Cecilia is innocent**, an **empty clause** was derived, which proves the innocence. Consequently, we add the fact: $\neg\text{guilty}(c)$ to the knowledge base as **Clause 18**.

5.5 Ann

Since **Eduardo** has been identified as **guilty** and **Brown**, **Cecilia**, and **Danielle** as **innocent**, we proceed to verify the remaining suspect by testing the hypothesis that **Ann is innocent**: $\neg\text{guilty}(a)$.

No	Derived Clause	Resolution Step / Description
33	$\neg\text{guilty}(a)$	Negated conclusion
34	$\neg\text{guilty}(a) \vee \text{guilty}(b) \vee \text{guilty}(c) \vee \text{guilty}(d) \vee \text{guilty}(e)$	Resolution of clauses 33 and 1
35	$\text{guilty}(a) \vee \text{guilty}(b) \vee \text{guilty}(c) \vee \text{guilty}(d) \vee \text{guilty}(e)$	Resolution of clauses 18 and 34
36	$\neg\text{girl}(a) \vee \text{guilty}(a) \vee \neg\text{girl}(\text{Z}) \vee \neg\text{girl}(\text{Z}) \vee \text{guilty}(e)$	Resolution of clauses 35 and 10
37	$\neg\text{girl}(a) \vee \neg\text{girl}(\text{Z}) \vee \neg\text{girl}(\text{Z}) \vee \text{guilty}(e)$	Resolution of clauses 36 and 2
38	$\neg\text{girl}(\text{Z}) \vee \neg\text{girl}(\text{Z}) \vee \text{guilty}(e)$	Resolution of clauses 37 and 2
39	$\neg\text{girl}(\text{Z}) \vee \text{guilty}(e)$	Resolution of clauses 38 and 38
40	$\square$	Resolution of Clauses 39 and 39

The resolution process starting from the negated hypothesis $\neg\text{guilty}(a)$ resulted in the derivation of an **empty clause**, which proves that **Ann is innocent**. Moreover, this result shows that the statement claiming that at least one of the girls is guilty is false.

6. Automated Solver Verification

To ensure the correctness of our solutions, we will verify it using automated SAT/SMT solvers, namely CPLJ All Solver and CP4J Program.

6.1 CPLJ SAT Solver

CPLJ All Solver is an implementation of the **Davis-Putnam-Logemann-Loveland (DPLL)** algorithm for checking the satisfiability of propositional logic formulas in conjunctive normal form (CNF). The algorithm uses **unit clause propagation** and **non-chronological backtracking** for efficient solution search.

To run the SAT solver correctly, our knowledge base is transformed into the format it accepts:

- The $\text{guilty}(x)$ predicate is removed, leaving only the literals represented as DIMACS clauses.
- A minus (-) indicates the negation of a literal.

Example Input:

```
a b c d e
-a -b
-a -c
-a -d
-a -e
-b -c
-b -d
-b -e
-c -d
-c -e
-d -e
a b c d e
-a -b -c -d -e
-a -b -c
-a -b -d
-a -b -e
-a -c -d
-a -c -e
-a -d -e
-b -c -d
-b -c -e
-b -d -e
-c -d -e
```

Solution:

```
SATISFIABLE e -a -b -d -c
```

6.2 CP4J Program

CP4J Program is a web interface for the **CP-SAT solver**, which allows checking the satisfiability of logical formulas written in the SAT-LIB language. It supports Boolean, integer, and other types of variables, enabling the formalization of more complex constraints.

The CP4J Program is more convenient than the normal SAT-LIB format:

```
int a, b, c, d, e;
bool[] guilty = [a, b, c, d, e];
bool[] girl = [a, b, c, d, e];
bool Z;
bool[] Z in [false, true];
bool[] Brown = [false, true];
bool[] Cecilia = [false, true];
bool[] Danielle = [false, true];
bool[] Eduardo = [false, true];

// ------ Clauses ------
constraint exists(Z in girl, Z);

// Two ones
count(guilty, 2) == 2;

// At most one
count(guilty) <= 1;
count(guilty) <= 1;
count(guilty) <= 1;
count(guilty) <= 1;
count(guilty) <= 1;

count(girl) <= 1;
count(girl) <= 1;
count(girl) <= 1;
count(girl) <= 1;
count(girl) <= 1;

// ------ Brown ------
Brown impl "b or e";
// Brown says: "The culprit is either Eduardo or Danielle"
count(b) <= count(e) + count(d);

// ------ Eduardo ------
Eduardo impl "e -> exists(Z in girl, Z)";
// Eduardo says: "The guilty person is a girl"
count(e) <= count(Z in girl);

// ------ Danielle ------
Danielle impl "d <-> (b -> -c)";
// Danielle says: "If Brown is guilty, then Cecilia is innocent"
count(d) <= count(b) + count(c);

// ------ Check ------
check(all);
list(guilty);
```

Output:

```
sat
[
  [false, false, false, false, true]
  [false, false, false, false, true]
  [false, false, false, false, true]
  [false, false, false, false, true]
  [false, false, false, false, true]
  [false, false, false, false, true]
  [false, false, false, false, true]
  [false, false, false, false, true]
  [false, false, false, false, true]
  [false, false, false, false, true]
  [false, false, false, false, true]
  [true, false, false, false, true]
  [true, false, false, false, true]
  [true
```